End-to-End CI/CD Pipeline for a Calculator REST API

CSE 816 Software Production Engineering, Mini-Project · M.Tech (CSE), Term I 2026-27 · Faculty: Prof. B. Thangaraju
Ivan Bhargava (IMT2023022) · Ivan.Bhargava@iiitb.ac.in

GitHub: github.com/Ivan825/calculator-devops Docker Hub: hub.docker.com/r/vortexfrenzy/calculator-devops

1. Overview

The application is a Calculator REST API written in Java 21 with Spring Boot 3.5 and built with Maven. It exposes `add`, `subtract`, `multiply` and `divide` under `/api/v1`, and a Spring Actuator health probe at `/actuator/health`. Operands are handled as `BigDecimal`, so decimal arithmetic is exact (0.1 + 0.2 = 0.3). Every client error (division by zero, a missing or non-numeric parameter, an out-of-range operand) returns HTTP 400 with the same JSON body: `timestamp, status, error, message, path`.

Around the application sits a pipeline that needs no manual steps. A push to `main` on GitHub is picked up by Jenkins, which builds, tests and packages the code, builds a Docker image tagged with the build number, pushes it to Docker Hub and redeploys the Docker Compose stack. Jenkins then checks the stack's health and emails the result.

2. Architecture

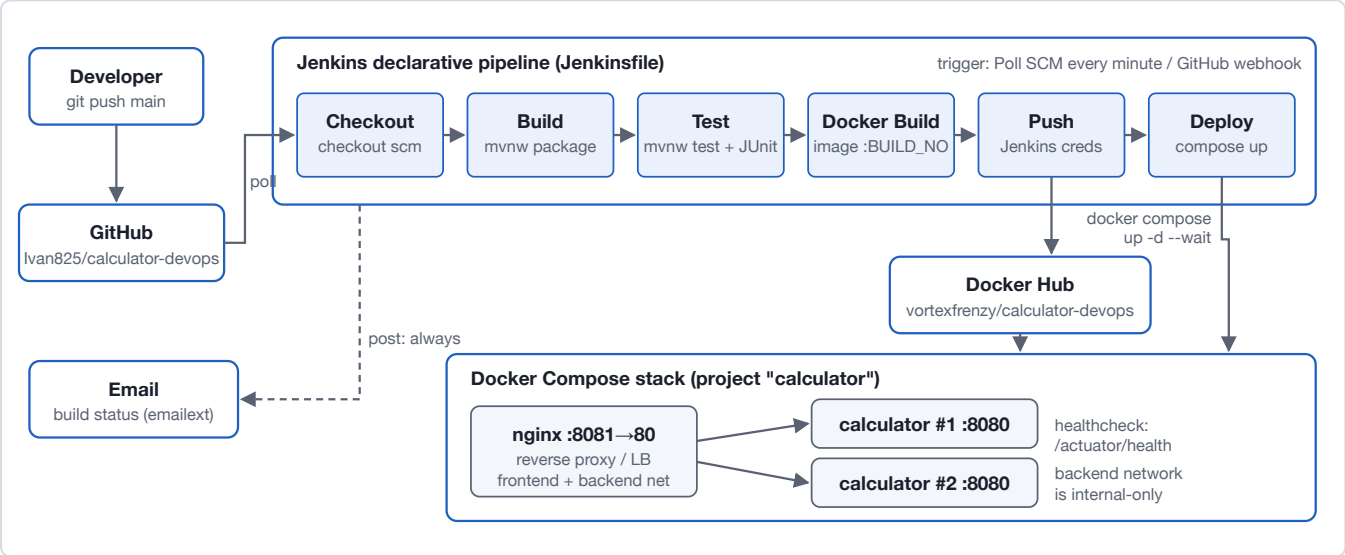


Figure 1: Pipeline and runtime architecture.

3. Pipeline stages

Stage	What it does	Build #1
Checkout	`checkout scm` fetches the commit that triggered the build. The job polls GitHub every minute (`pollSCM`), and a `githubPush()` trigger is also declared for a webhook.	2.0 s
Build	`./mvnw -B clean package -DskipTests` compiles the code into an executable JAR, which is archived and fingerprinted. The Maven wrapper means Jenkins needs no Maven install of its own.	11 s
Test	`./mvnw -B test` runs 33 JUnit 5 tests: unit tests for every operation including divide by zero, and `@SpringBootTest` + MockMvc integration tests for each endpoint, each error case and the health check. Surefire XML is published with `junit`. A failing test makes Maven exit non-zero, which fails the build.	7.3 s
Docker Build	Builds `vortexfrenzy/calculator-devops:<BUILD_NUMBER>` and `:latest` from the multi-stage Dockerfile.	15 s

Push	`withCredentials` binds the Docker Hub username and access token from the Jenkins credential store (ID `dockerhub-credentials`). Login uses `--password-stdin`, both tags are pushed and `docker logout` always runs afterwards. Nothing secret is in the repository.	1 min 8 s
Deploy	`docker compose up -d --no-build --remove-orphans --wait` rolls the stack to the new tag and blocks until every container is healthy. A smoke test then calls `/actuator/health` and `/api/v1/add` through Nginx. If deployment fails, `docker compose ps` and the container logs are printed.	20 s
Post	Prunes all but the last 5 numbered image tags and dangling layers, then `emailext` emails the result (SUCCESS or FAILURE) with links to the console and test report. Email goes through Gmail SMTP using credentials stored in Jenkins.	6.2 s

Re-runnable and idempotent. `disableConcurrentBuilds()` stops two deploys from racing. Compose recreates only the containers whose image or configuration changed, so re-running a build leaves a healthy deployment as it is; a second `up` with the same tag recreated nothing.

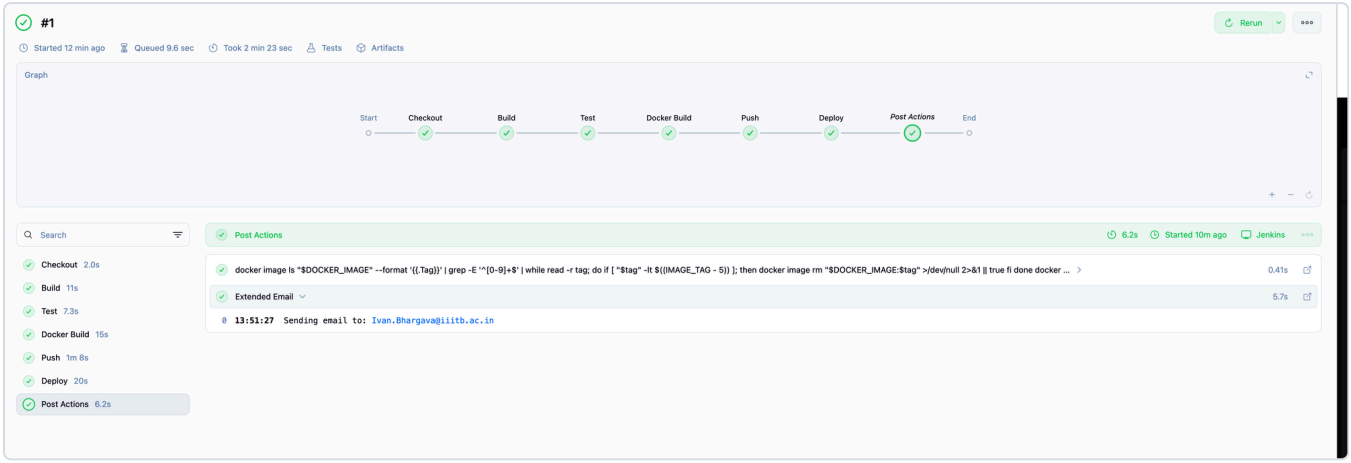
Containerization and deployment

- **Dockerfile (multi-stage).** The build stage (`maven:3.9.11-eclipse-temurin-21`) copies `pom.xml` and runs `dependency:go-offline` before copying `src/`, so the dependency layer stays cached until the POM changes. The runtime stage (`eclipse-temurin:21-jre-alpine`) copies only the JAR, runs as the non-root user `spring`, declares `EXPOSE 8080` and has a `HEALTHCHECK`. A `.dockerignore` keeps `target/`, `.git/` and IDE files out of the build context.
- **docker-compose.yml.** Two `calculator` replicas (health check on `/actuator/health`, `restart: unless-stopped`, environment variables `SERVER_PORT`, `APP_VERSION`, `JAVA_OPTS`) sit on an internal `backend` network. An `nginx` reverse proxy, which waits for the API to be healthy and has its own health check, is the only service on the `frontend` network and publishes host port 8081. One `docker compose up` starts the whole stack.
- **Git workflow.** Development happened in steps on `feature/calculator-api`, `feature/containerization` and `feature/jenkins-pipeline`, each merged into `main` with a merge commit. A `.gitignore` covers Maven, IDE files and secrets.

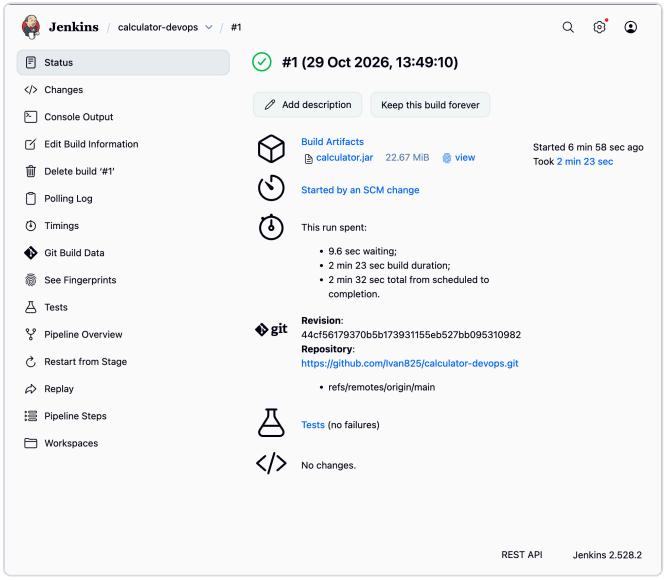
4. Challenges faced

- **Nginx sent every request to one replica.** My first configuration used `proxy_pass` with a variable and Docker's DNS resolver. Nginx resolved the service name once per DNS TTL and kept using the first address, so the `X-Upstream-Server` header never changed. I switched to an `upstream` block with a shared-memory `zone` and `server calculator:8080 resolve` (available in open-source Nginx 1.27.3 and later). Requests now alternate between both replicas, and Nginx picks up new IPs when Compose recreates the containers.
- **Port conflict with Jenkins.** Jenkins already listens on 8080, so the stack publishes Nginx on 8081, set in `.env`.
- **Jenkins runs with a minimal PATH.** Homebrew's Jenkins runs as a macOS launch agent whose PATH doesn't include Docker Desktop's `/usr/local/bin`. The Jenkinsfile prepends it in its `environment` block, and the Maven wrapper avoids needing a Jenkins-managed Maven install.
- **No public URL for a webhook.** Jenkins listens only on `localhost`, so GitHub can't deliver a webhook to it without a tunnel such as ngrok. The job polls GitHub every minute instead, which the brief accepts. The `githubPush()` trigger is still declared, so adding a tunnel and a webhook would make builds start instantly.
- **Credential ID rejected.** Jenkins reported "Unacceptable characters" for `dockerhub-credentials`. A pasted invisible leading space was the cause; retyping the ID fixed it.
- **Number formatting and abuse.** `BigDecimal.stripTrailingZeros()` serialised 10 as `1E+1`, which `spring.jackson.generator.write-bigdecimal-as-plain=true` fixed. Inputs like `1e999999999` could force huge computations, so operands are limited to 50 significant digits and an exponent within ±1000, with a 400 response outside that.

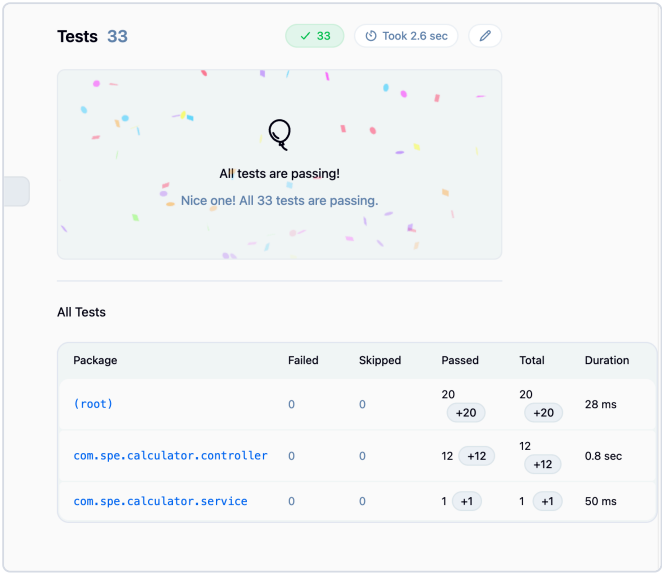
Appendix: Screenshots



A1. Jenkins build #1: every stage green (Checkout, Build, Test, Docker Build, Push, Deploy, Post Actions).



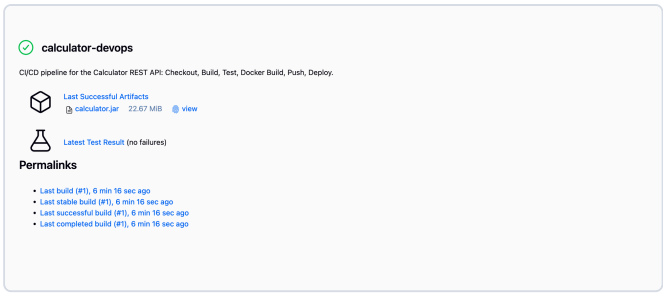
A2. Build #1, started by an SCM change (poll), revision and artifacts.



A3. Published JUnit results: 33 tests, 0 failures.

```
13:51:26  calculator-nginx-1        nginx:1.28-alpine
"/docker-entrypoint.…"   nginx        18 seconds ago   Up 11
seconds (healthy)   0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
13:51:26  + exit 0
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] sh
13:51:26  + docker image ls vortexfrenzy/calculator-devops --
format '{{.Tag}}'
13:51:26  + grep -E '^[0-9]+$'
13:51:26  + read -r tag
13:51:26  + '[' 1 -lt -4 ']'
13:51:26  + read -r tag
13:51:26  + docker image prune -f
[Pipeline] emailext
13:51:27  Sending email to: Ivan.Bhargava@iiitb.ac.in
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // timeout
[Pipeline] }
[Pipeline] // timestamps
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

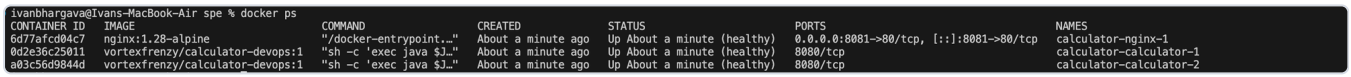
A4. Console: healthy stack, email sent, `Finished: SUCCESS`.



A5. Job page: archived `calculator.jar`, latest test result.

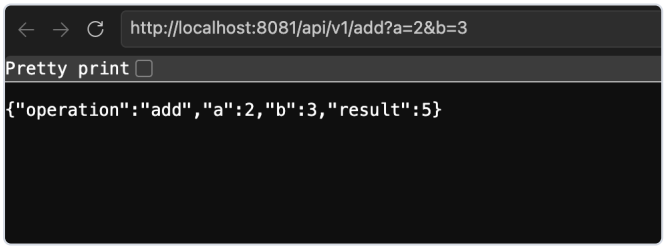


A6. Email notification of the build status.

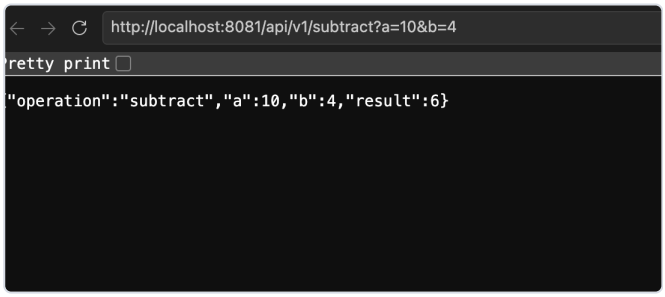


A7. `docker ps`: Nginx on 0.0.0.0:8081 and two healthy calculator replicas running image tag `:1`.

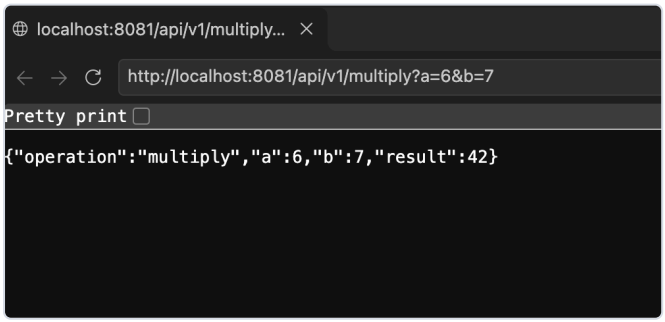
Sample API responses (through Nginx on port 8081)



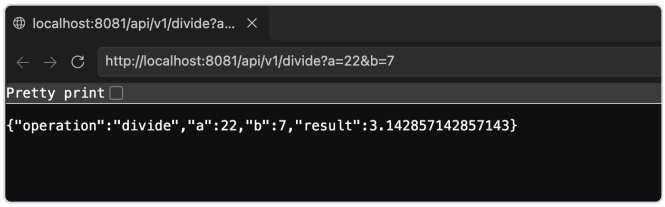
A8. `/api/v1/add?a=2&b=3`



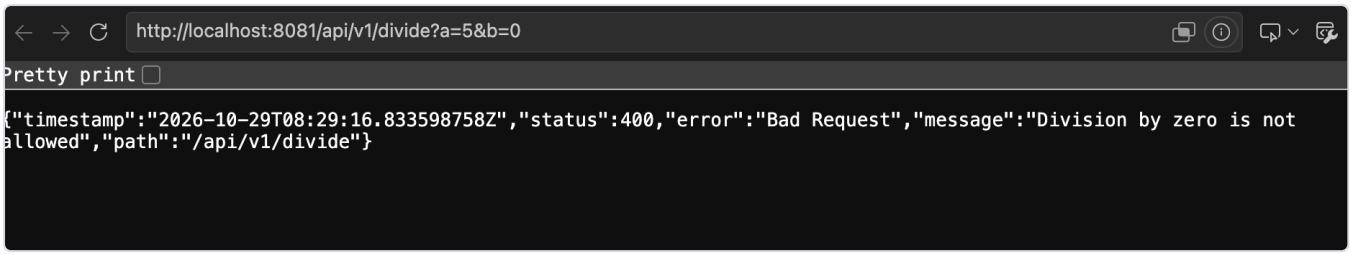
A9. `/api/v1/subtract?a=10&b=4`



A10. `/api/v1/multiply?a=6&b=7`



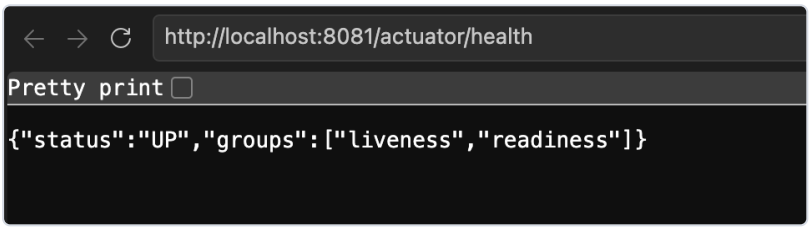
A11. `/api/v1/divide?a=22&b=7`



A12. `/api/v1/divide?a=5&b=0` returns HTTP 400 with error JSON.



A13. `/api/v1/add?a=abc&b=2` returns HTTP 400 for non-numeric input.



A14. `/actuator/health` returns UP.